JAXB — JAVA API FOR XML BINDING

Hands-On Lab Workbook

Platform: Java 11 LTS | Tomcat 9.0 | JAXB 2.3.x | Maven 3.8+
Package namespace: javax.xml.bind.* (NOT jakarta.xml.bind.*)
Binding file namespace: http://java.sun.com/xml/ns/jaxb version="2.1"
Starter project: jaxb-lab-starter.zip (import as Maven project)
Labs: 4 Labs — approx. 3.5 hours total

© Training Materials — All Rights Reserved

Lab Overview and Prerequisites

Package Namespace — Important

All code uses javax.xml.bind.* (NOT jakarta.xml.bind.*) — correct for Java 11 / Tomcat 9.
All binding files use JAXB 2.x namespace: http://java.sun.com/xml/ns/jaxb version="2.1"
Do NOT use https://jakarta.ee/xml/ns/jaxb — that is for Tomcat 10 / Jakarta EE 9+ only.
import javax.xml.bind.JAXBContext; import javax.xml.bind.annotation.*;
import javax.xml.bind.annotation.adapters.XmlAdapter;

Labs in This Workbook

Lab	Title	Covers	Section	Time
1	Schema-First with xjc	Write library.xsd, run generate-sources	Sec 6	45 min
2	Annotating Java Classes	@XmlRootElement, @XmlAttribute, @XmlElementWrapper	Sec 2–3	50 min
3	Marshal, Unmarshal & Validate	OrderXmlUtil, JAXBContext, schema validation	Sec 2 & 5	45 min
4	JAXB in a Servlet	JaxbContextListener, OrderServlet, StAX batch	Sec 2 & 5	55 min

Quick Start

1. Unzip jaxb-lab-starter.zip and import as a Maven project.
2. Run: mvn clean package -DskipTests — build must succeed.
3. Run: mvn test -Dtest=SmokeTest — all 13 baseline tests must PASS.
4. All imports must use javax.xml.bind.*, NOT jakarta.xml.bind.*
5. All binding files (.xjb) must use xmlns:jaxb="http://java.sun.com/xml/ns/jaxb" version="2.1"

© Training Materials — All Rights Reserved

LAB 1

Schema-First Development with xjc

Section: Section 6

45 min

Objectives

- Write a complete XSD schema for a library domain
- Run xjc via mvn generate-sources to produce Java binding classes
- Understand the generated code (ObjectFactory, package-info.java)
- Use generated classes to marshal and unmarshal a Library object
- Apply binding customisations to rename generated classes

Before you start

The skeleton XSD is at src/main/xsd/library.xsd — open it now.
The file contains TODO comments for each section you must complete.
After each task, run: mvn generate-sources
Check target/generated-sources/jaxb/com/example/library/model/ for output.
If xjc reports an error, fix the XSD and run generate-sources again.

Task 1.1 — Understand the Target XML

Your completed schema must validate this sample document. Read it carefully before writing any XSD:

```
<?xml version="1.0" encoding="UTF-8"?>
<library xmlns="http://example.com/library"
         name="City Central Library"
         totalBooks="2">
  <book isbn="978-0-13-235088-4" genre="TECHNOLOGY">
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
    <yearPublished>2008</yearPublished>
    <availableCopies>3</availableCopies>
  </book>
  <book isbn="978-0-13-468599-1" genre="TECHNOLOGY">
    <title>Effective Java</title>
    <author>Joshua Bloch</author>
    <yearPublished>2018</yearPublished>
    <availableCopies>2</availableCopies>
  </book>
</library>
```

Task 1.2 — Complete library.xsd

Open src/main/xsd/library.xsd and complete each TODO section:

1. Task 1.2a — Declare the global <library> root element of type LibraryType
2. Task 1.2b — Define LibraryType: sequence of book elements, name attribute (required), totalBooks attribute (optional)
3. Task 1.2c — Define BookType: child elements title, author, yearPublished, availableCopies; attributes isbn (required) and genre (optional)
4. Task 1.2d — Define GenreType as a simpleType restriction with values: TECHNOLOGY, FICTION, NON_FICTION, SCIENCE, HISTORY, BIOGRAPHY, OTHER

© Training Materials — All Rights Reserved

Paste your completed xs:complexType for LibraryType here:

Task 1.3 — Generate Java Classes

```
mvn generate-sources

# Expected output in target/generated-
sources/jaxb/com/example/library/model/:
#   LibraryType.java   — or Library.java after Task 1.5
#   BookType.java      — or Book.java after Task 1.5
#   GenreType.java     — or Genre.java after Task 1.5
#   ObjectFactory.java — factory methods for all types
#   package-info.java  — namespace declaration
```

List all generated files and describe what each one contains:

Task 1.4 — Marshal a Library to XML

Create a scratch main class or JUnit test to marshal a Library object. Use ObjectFactory to create instances — do NOT call new LibraryType() directly:

```
import com.example.library.model.*;
import javax.xml.bind.JAXBContext;
import javax.xml.bind.Marshaller;

ObjectFactory factory = new ObjectFactory();

BookType book = factory.createBookType();
book.setTitle("Clean Code");
book.setAuthor("Robert C. Martin");
book.setYearPublished(2008);
book.setAvailableCopies(3);
book.setIsbn("978-0-13-235088-4");
book.setGenre(GenreType.TECHNOLOGY);

LibraryType lib = factory.createLibraryType();
lib.setName("City Central Library");
lib.setTotalBooks(BigInteger.ONE);
lib.getBook().add(book);

JAXBContext ctx = JAXBContext.newInstance(LibraryType.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(factory.createLibrary(lib), System.out);
```

© Training Materials — All Rights Reserved

Paste the XML output here:

Task 1.5 — Apply Binding Customisations

Open src/main/xsd/bindings.xjb. Uncomment the three TODO blocks to rename LibraryType→Library, BookType→Book, GenreType→Genre.

Binding file namespace — important

bindings.xjb uses the JAXB 2.x namespace and version. The file already has:
xmlns:jaxb="http://java.sun.com/xml/ns/jaxb" version="2.1"

Do NOT change this to the Jakarta EE namespace:
https://jakarta.ee/xml/ns/jaxb version="3.0"
That namespace is for Tomcat 10 only and will cause the plugin to fail.

Then in pom.xml, remove the <bindingIncludes> block and uncomment the <bindingDirectory> block. Run mvn clean generate-sources and verify the class names have changed:

```
# After completing bindings.xjb and updating pom.xml:
mvn clean generate-sources

# Expected generated classes (renamed):
#   Library.java
#   Book.java
#   Genre.java  (enum)
#   ObjectFactory.java
#   package-info.java
```

Q1

Why do we use ObjectFactory.createLibrary(lib) rather than just m.marshal(lib, out)? What would happen if you passed the LibraryType instance directly to marshal()?

Your answer:

Lab 1 Checkpoint

© Training Materials — All Rights Reserved

- ✓ mvn generate-sources produces 5 files in target/generated-sources/jaxb/
- ✓ Generated XML matches the target document in Task 1.1
- ✓ After Task 1.5: classes are named Library, Book, Genre (not LibraryType etc.)
- ✓ bindings.xjb uses xmlns:jaxb="http://java.sun.com/xml/ns/jaxb" version="2.1"

© Training Materials — All Rights Reserved

LAB 2

Annotating Java Classes — Order Domain

Section: Sections 2 & 3

50 min

Objectives

- Apply @XmlRootElement, @XmlAccessorType, and @XmlType to the Order class
- Use @XmlAttribute for orderId and status, @XmlElement for child elements
- Apply @XmlElementWrapper to produce a <lines> wrapper around the collection
- Implement LocalDateAdapter to marshal LocalDate as a yyyy-MM-dd string
- Use @XmlTransient to exclude internalRef from the XML output

Package Namespace — Important

All code uses javax.xml.bind.* (NOT jakarta.xml.bind.*) — correct for Java 11 / Tomcat 9.
All binding files use JAXB 2.x namespace: http://java.sun.com/xml/ns/jaxb version="2.1"
Do NOT use https://jakarta.ee/xml/ns/jaxb — that is for Tomcat 10 / Jakarta EE 9+ only.
import javax.xml.bind.JAXBContext; import javax.xml.bind.annotation.*;
import javax.xml.bind.annotation.adapters.XmlAdapter;

Task 2.1 — Read the Target XML

Before adding any annotations, study the target XML in the Javadoc of Order.java. Every element, attribute, and structure in the target must be produced by your annotations. Confirm you understand:

- Which fields become XML attributes (orderId, status)
- Which fields become child XML elements (customerName, customerEmail, etc.)
- How the lines collection is wrapped inside <lines>
- Which field must be excluded (internalRef)

Task 2.2 — Annotate Order at Class and Field Level

Open Order.java. Add the class-level annotations and then the field-level annotations for orderId and status. The commented-out import block at the top of the file shows the correct javax.* imports to add:

```
// Class-level annotations to add:
@XmlRootElement(name = "order",
                namespace = "http://example.com/shop/orders")
@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(propOrder = {
    "customerName", "customerEmail", "orderDate",
    "deliveryAddress", "lines", "totalAmount", "notes"
})
public class Order {

    @XmlAttribute(required = true)
    private String orderId;

    @XmlAttribute(required = true)
    private OrderStatus status;
    // ... remaining fields
}
```

© Training Materials — All Rights Reserved

Quick marshal test — run this to check your annotations so far:

```
import javax.xml.bind.JAXBContext;
import javax.xml.bind.Marshaller;

Order order = new Order();
order.setOrderId("ORD-TEST");
order.setStatus(OrderStatus.NEW);
order.setCustomerName("Test");

JAXBContext ctx = JAXBContext.newInstance(Order.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(order, System.out);

// Expected: orderId and status appear as XML ATTRIBUTES
// i.e.  <order orderId="ORD-TEST" status="NEW">
// NOT:  <orderId>ORD-TEST</orderId>
```

Task 2.3 — Annotate Child Elements, Address, and OrderLine

Add @XmlElement to customerName, customerEmail, orderDate, deliveryAddress, and notes. Add @XmlElementWrapper + @XmlElement to the lines list. Then annotate Address.java and OrderLine.java:

```
// In Order.java — lines collection:
@XmlElementWrapper(name = "lines")
@XmlElement(name = "line")
private List<OrderLine> lines = new ArrayList<>();

// In Address.java — class level:
@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(propOrder = { "street", "city", "postCode", "country" })
public class Address { ... }

// In OrderLine.java — class level:
@XmlAccessorType(XmlAccessType.FIELD)
@XmlType(propOrder = {
    "productName", "quantity", "unitPrice", "lineTotal" })
public class OrderLine { ... }

// sku is an ATTRIBUTE on each <line> element:
@XmlAttribute(required = true)
private String sku;
```

Task 2.4 — Annotate OrderStatus Enum

```
import javax.xml.bind.annotation.XmlEnum;
import javax.xml.bind.annotation.XmlType;

@XmlType(name = "OrderStatusType")
@XmlEnum
public enum OrderStatus {
    NEW, PROCESSING, SHIPPED, DELIVERED, CANCELLED
}
```

Task 2.5 — Implement LocalDateAdapter and Apply It

© Training Materials — All Rights Reserved

Open adapter/LocalDateAdapter.java. Complete the two methods. Both import from javax.xml.bind.annotation.adapters.XmlAdapter:

```
import javax.xml.bind.annotation.adapters.XmlAdapter;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

public class LocalDateAdapter extends XmlAdapter<String, LocalDate> {

    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ISO_LOCAL_DATE;

    @Override
    public LocalDate unmarshal(String value) {
        if (value == null || value.isBlank()) return null;
        return LocalDate.parse(value.trim(), FMT);
    }

    @Override
    public String marshal(LocalDate value) {
        return value == null ? null : value.format(FMT);
    }
}

// Apply on Order.orderDate field:
@XmlElement(required = true)
@XmlJavaTypeAdapter(LocalDateAdapter.class)
private LocalDate orderDate;
```

Task 2.6 — Full Marshal Test

Run OrderFixtures.createSampleOrder() through your annotated classes and verify the output matches the target XML in Order.java's Javadoc:

```
Order order = OrderFixtures.createSampleOrder();
JAXBContext ctx = JAXBContext.newInstance(Order.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(order, System.out);

// Verify ALL of the following:
// ✓  orderId="ORD-2024-001" appears as an ATTRIBUTE
// ✓  status="PROCESSING" appears as an ATTRIBUTE
// ✓  <orderDate>2024-06-15</orderDate>  (yyyy-MM-dd format)
// ✓  <lines><line sku="BK-001">...</line></lines>
// ✓  <unitPrice>29.99</unitPrice>  (2 decimal places)
// ✓  internalRef does NOT appear anywhere in the output
```

Paste your XML output here:

© Training Materials — All Rights Reserved

Lab 2 Checkpoint

- ✓ mvn test -Dtest=SmokeTest still all green
- ✓ orderId and status are XML attributes, not child elements
- ✓ <lines> wrapper element appears with <line> children
- ✓ orderDate is formatted as 2024-06-15 (not as a number or array)
- ✓ internalRef does NOT appear in the marshalled XML

© Training Materials — All Rights Reserved

LAB 3

Marshal, Unmarshal, and Schema Validation

Section: Sections 2 & 5

45 min

Objectives

- Implement the three methods in OrderXmlUtil using javax.xml.bind.JAXBContext
- Write the OrderAnnotationTest test implementations
- Attach a Schema to an Unmarshaller to enable validation during unmarshalling
- Write a ValidationEventHandler that fails on ERROR and collects warnings

Key types — all from javax.*

javax.xml.bind.JAXBContext — create once, thread-safe
javax.xml.bind.Marshaller — create per operation, NOT thread-safe
javax.xml.bind.Unmarshaller — create per operation, NOT thread-safe
javax.xml.bind.ValidationEvent — severity constants: WARNING=0, ERROR=1, FATAL_ERROR=2
javax.xml.validation.Schema — create once from SchemaFactory, thread-safe
javax.xml.validation.SchemaFactory — use XMLConstants.W3C_XML_SCHEMA_NS_URI

Task 3.1 — Implement OrderXmlUtil.toXml() and toFile()

Open xml/OrderXmlUtil.java. The CTX and SCHEMA static fields are already initialised. Implement the two marshal methods:

```
// toXml() — marshal to a String
public static String toXml(Order order) throws JAXBException {
    Marshaller m = CTX.createMarshaller();
    m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
    m.setProperty(Marshaller.JAXB_ENCODING, "UTF-8");
    StringWriter sw = new StringWriter();
    m.marshal(order, sw);
    return sw.toString();
}

// toFile() — marshal to a file path
public static void toFile(Order order, Path target)
        throws JAXBException, IOException {
    Marshaller m = CTX.createMarshaller();
    m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
    try (BufferedWriter w = Files.newBufferedWriter(
            target, StandardCharsets.UTF_8)) {
        m.marshal(order, w);
    }
}
```

Task 3.2 — Implement OrderXmlUtil.fromXml()

```
// fromXml() — unmarshal from a String, no validation
public static Order fromXml(String xml) throws JAXBException {
    Unmarshaller u = CTX.createUnmarshaller();
    return (Order) u.unmarshal(new StringReader(xml));
}
```

© Training Materials — All Rights Reserved

Task 3.3 — Implement OrderXmlUtil.fromXmlValidated() and the Test Class

Implement fromXmlValidated() with schema validation attached. Then implement each TODO test method in OrderAnnotationTest.java:

```
// fromXmlValidated() — unmarshal with schema validation
public static Order fromXmlValidated(InputStream stream)
        throws JAXBException {
    Unmarshaller u = CTX.createUnmarshaller();
    u.setSchema(SCHEMA);    // attach the pre-compiled schema
    u.setEventHandler(event -> {
        if (event.getSeverity() >= ValidationEvent.ERROR)
            throw new RuntimeException(
                "Validation error: " + event.getMessage());
        return true;        // continue on WARNING
    });
    return (Order) u.unmarshal(stream);
}

// Example test implementations in OrderAnnotationTest:
@Test
void marshal_orderIdIsAnAttribute() throws Exception {
    String xml = marshal(OrderFixtures.createSampleOrder());
    assertTrue(xml.contains("orderId=\"ORD-2024-001\""),
        "orderId must be an XML attribute");
}

@Test
void marshal_internalRefIsAbsent() throws Exception {
    String xml = marshal(OrderFixtures.createSampleOrder());
    assertFalse(xml.contains("internalRef"),
        "internalRef must not appear in XML");
    assertFalse(xml.contains("INTERNAL-REF-001"),
        "internalRef value must not appear in XML");
}
```

Q1

ValidationEvent has three severity levels: WARNING (0), ERROR (1), FATAL_ERROR (2). Why should your handler check severity >= ValidationEvent.ERROR rather than severity == ValidationEvent.FATAL_ERROR?

Your answer:

Q2

u.setSchema(schema) and u.setEventHandler(handler) are both required for schema validation to work. What happens if you call only setEventHandler() without setSchema()?

Your answer:

© Training Materials — All Rights Reserved

Lab 3 Checkpoint

- ✓ mvn test -Dtest=OrderAnnotationTest — all tests pass
- ✓ toXml() produces correct XML with orderId as attribute
- ✓ fromXml() round-trips an Order preserving all fields
- ✓ fromXmlValidated() throws on XML missing required orderId attribute

© Training Materials — All Rights Reserved

LAB 4

JAXB in a Servlet — HTTP XML API and Batch Processing

Section: Sections 2 & 5

55 min

Objectives

- Implement JaxbContextListener to initialise JAXBContext once at startup
- Implement OrderServlet.doPost() to accept XML, validate, store, and respond
- Implement OrderServlet.doGet() to marshal a stored order to XML
- Implement OrderBatchProcessor using the StAX + JAXB partial-unmarshal pattern
- Apply XXE prevention to every XMLInputFactory

Servlet API namespace reminder

Tomcat 9 uses the javax.servlet.* packages (NOT jakarta.servlet.*).
Your imports must be:

```
import javax.servlet.annotation.WebServlet;
import javax.servlet.annotation.WebListener;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.ServletContextListener;
import javax.servlet.ServletContextEvent;
```

Task 4.1 — Implement JaxbContextListener

Open web/JaxbContextListener.java. Implement contextInitialized() — the TODO comment block shows exactly what to do:

```
@WebListener
public class JaxbContextListener implements ServletContextListener {

    public static final String JAXB_CTX_ATTR    = "jaxbContext";
    public static final String JAXB_SCHEMA_ATTR = "jaxbSchema";

    @Override
    public void contextInitialized(ServletContextEvent sce) {
        try {
            // 1. Create JAXBContext
            JAXBContext ctx = JAXBContext.newInstance(Order.class);
            sce.getServletContext().setAttribute(JAXB_CTX_ATTR, ctx);

            // 2. Compile Schema
            SchemaFactory sf = SchemaFactory.newInstance(
                XMLConstants.W3C_XML_SCHEMA_NS_URI);
            sf.setProperty(XMLConstants.ACCESS_EXTERNAL_DTD,    "");
            sf.setProperty(XMLConstants.ACCESS_EXTERNAL_SCHEMA,  "");
            Schema schema = sf.newSchema(
                getClass().getResource("/xsd/order-schema.xsd"));
            sce.getServletContext().setAttribute(JAXB_SCHEMA_ATTR, schema);

            System.out.println("[JAXB] Context and schema initialised OK.");
        } catch (Exception e) {
            throw new RuntimeException(
```

© Training Materials — All Rights Reserved

```
                "JAXB initialisation failed", e);
        }
    }
}
```

Task 4.2 — Implement OrderServlet.doPost()

Open web/OrderServlet.java. Implement doPost() — unmarshal the XML request body, validate it, store the order, and respond 201 with the marshalled order:

```
@Override
protected void doPost(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
    try {
        JAXBContext ctx    = getCtx();
        Schema schema      = getSchema();
        Unmarshaller u     = ctx.createUnmarshaller();
        u.setSchema(schema);
        u.setEventHandler(event -> {
            if (event.getSeverity() >= ValidationEvent.ERROR)
                throw new RuntimeException(event.getMessage());
            return true;
        });
        Order order = (Order) u.unmarshal(req.getInputStream());
        store.put(order.getOrderId(), order);

        resp.setStatus(HttpServletResponse.SC_CREATED);   // 201
        resp.setContentType("text/xml;charset=UTF-8");
        Marshaller m = ctx.createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        m.marshal(order, resp.getWriter());

    } catch (JAXBException e) {
        resp.sendError(HttpServletResponse.SC_BAD_REQUEST, e.getMessage());
    }
}
```

Test with curl once deployed to Tomcat:

```
curl -v -X POST http://localhost:8080/jaxb-lab/api/orders \
     -H 'Content-Type: text/xml' \
     --data-binary @src/test/resources/xml/sample-order.xml

# Expected: HTTP 201, XML response body with the stored order
```

Task 4.3 — Implement OrderServlet.doGet()

```
@Override
protected void doGet(HttpServletRequest req, HttpServletResponse resp)
        throws IOException {
    // Extract orderId from path: /api/orders/{orderId}
    String pathInfo = req.getPathInfo();  // e.g. "/ORD-2024-001"
    if (pathInfo == null || pathInfo.length() <= 1) {
        resp.sendError(HttpServletResponse.SC_BAD_REQUEST,
            "Order ID required in path"); return;
    }
    String orderId = pathInfo.substring(1);
    Order order = store.get(orderId);
    if (order == null) {
```

© Training Materials — All Rights Reserved

```
        resp.sendError(HttpServletResponse.SC_NOT_FOUND,
            "Order not found: " + orderId); return;
    }
    resp.setContentType("text/xml;charset=UTF-8");
    try {
        Marshaller m = getCtx().createMarshaller();
        m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
        m.marshal(order, resp.getWriter());
    } catch (JAXBException e) {
        resp.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
            e.getMessage());
    }
}
```

```
# After POST, retrieve the order:
curl http://localhost:8080/jaxb-lab/api/orders/ORD-2024-001
# Expected: HTTP 200, XML body

# Non-existent order:
curl -v http://localhost:8080/jaxb-lab/api/orders/DOES-NOT-EXIST
# Expected: HTTP 404
```

Task 4.4 — Implement OrderBatchProcessor

Open batch/OrderBatchProcessor.java. Complete the constructor (add XXE prevention) and implement the process() method using the StAX + JAXB partial-unmarshal pattern:

```
// Constructor — add XXE prevention
public OrderBatchProcessor(JAXBContext ctx) {
    this.ctx = ctx;
    this.xif = XMLInputFactory.newInstance();
    // SECURITY — disable external entity processing
    xif.setProperty(
        XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false);
    xif.setProperty(XMLInputFactory.SUPPORT_DTD, false);
}

// process() — StAX cursor + JAXB partial unmarshal
public void process(Path batchFile, Consumer<Order> handler)
        throws IOException, XMLStreamException, JAXBException {
    try (InputStream is = Files.newInputStream(batchFile)) {
        XMLStreamReader xsr = xif.createXMLStreamReader(is);
        Unmarshaller u = ctx.createUnmarshaller();
        while (xsr.hasNext()) {
            int event = xsr.next();
            if (event == XMLStreamConstants.START_ELEMENT
                    && "order".equals(xsr.getLocalName())) {
                JAXBElement<Order> je =
                    u.unmarshal(xsr, Order.class);
                handler.accept(je.getValue());
            }
        }
        xsr.close();
    }
}
```

Task 4.5 — Test the Batch Processor

Create a batch XML file containing three orders and run the processor:

© Training Materials — All Rights Reserved

```
// Generate the batch file
Path batchFile = Path.of("target/test-batch.xml");
try (BufferedWriter w = Files.newBufferedWriter(batchFile)) {
    w.write("<?xml version=\"1.0\" encoding=\"UTF-8\"?>");
    w.write("<orders>");
    for (int i = 1; i <= 3; i++) {
        Order o = OrderFixtures.createMinimalOrder(
            String.format("ORD-BATCH-%03d", i));
        Marshaller m = OrderXmlUtil.CTX.createMarshaller();
        m.setProperty(Marshaller.JAXB_FRAGMENT, Boolean.TRUE);
        m.marshal(o, w);
    }
    w.write("</orders>");
}

// Process the batch file
OrderBatchProcessor processor =
    new OrderBatchProcessor(OrderXmlUtil.CTX);
List<String> ids = new ArrayList<>();
processor.process(batchFile, order -> ids.add(order.getOrderId()));

System.out.println("Processed: " + ids);
// Expected: [ORD-BATCH-001, ORD-BATCH-002, ORD-BATCH-003]
```

Q1

Why must Marshaller.JAXB_FRAGMENT be set to true when marshalling individual orders into the batch file? What happens if you omit this property?

Your answer:

Q2

After calling u.unmarshal(xsr, Order.class) inside the StAX loop, where is the XMLStreamReader cursor positioned? Why does this matter for the next iteration of the loop?

Your answer:

Lab 4 Checkpoint

- ✓ POST /api/orders with valid XML returns 201 and the echoed order
- ✓ POST /api/orders with invalid XML (missing orderId) returns 400
- ✓ GET /api/orders/{id} returns 200 with the stored order XML
- ✓ GET /api/orders/unknown returns 404
- ✓ Batch processor correctly calls handler for each <order> element

© Training Materials — All Rights Reserved

✓ XXE properties set on XMLInputFactory before any parsing

© Training Materials — All Rights Reserved

Model Answers — Key Code

Reference implementations. Consult only after attempting each lab.

Lab 1 — Marshal a Generated Library (after Task 1.5 renaming)

```
ObjectFactory factory = new ObjectFactory();
Book book = factory.createBook();
book.setTitle("Clean Code");
book.setIsbn("978-0-13-235088-4");
book.setGenre(Genre.TECHNOLOGY);
book.setYearPublished(2008);
book.setAvailableCopies(3);
book.setAuthor("Robert C. Martin");

Library lib = factory.createLibrary();
lib.setName("City Central Library");
lib.setTotalBooks(BigInteger.ONE);
lib.getBook().add(book);

JAXBContext ctx = JAXBContext.newInstance(Library.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(factory.createLibrary(lib), System.out);
```

Lab 1.5 — Completed bindings.xjb (correct namespace)

```
<!-- Use JAXB 2.x namespace — NOT the Jakarta EE namespace -->
<jaxb:bindings
    xmlns:jaxb="http://java.sun.com/xml/ns/jaxb"
    xmlns:xs="http://www.w3.org/2001/XMLSchema"
    version="2.1">

  <jaxb:bindings schemaLocation="library.xsd" node="/xs:schema">
    <jaxb:bindings node="//xs:complexType[@name='LibraryType']">
      <jaxb:class name="Library"/>
    </jaxb:bindings>
    <jaxb:bindings node="//xs:complexType[@name='BookType']">
      <jaxb:class name="Book"/>
    </jaxb:bindings>
    <jaxb:bindings node="//xs:simpleType[@name='GenreType']">
      <jaxb:typesafeEnumClass name="Genre"/>
    </jaxb:bindings>
  </jaxb:bindings>

</jaxb:bindings>
```

Lab 2 — LocalDateAdapter

```
import javax.xml.bind.annotation.adapters.XmlAdapter;

public class LocalDateAdapter extends XmlAdapter<String, LocalDate> {
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ISO_LOCAL_DATE;
```

© Training Materials — All Rights Reserved

```
    @Override
    public LocalDate unmarshal(String v) {
        return (v == null || v.isBlank()) ? null
            : LocalDate.parse(v.trim(), FMT);
    }

    @Override
    public String marshal(LocalDate v) {
        return v == null ? null : v.format(FMT);
    }
}
```

Lab 3 — fromXmlValidated()

```
public static Order fromXmlValidated(InputStream stream)
        throws JAXBException {
    Unmarshaller u = CTX.createUnmarshaller();
    u.setSchema(SCHEMA);
    u.setEventHandler(event -> {
        if (event.getSeverity() >= ValidationEvent.ERROR)
            throw new RuntimeException(
                "Validation failed: " + event.getMessage());
        return true;
    });
    return (Order) u.unmarshal(stream);
}
```

© Training Materials — All Rights Reserved